

LAPORAN PRAKTIKUM 8
STRUKTUR DATA
IMPLEMENTASI *ABSTRACT DATA TYPE (ADT)*
SORTING MENGGUNAKAN JAVA



Disusun Oleh:

Naysha Aprilia Rizki

2511531014

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul **“Laporan Praktikum 8 Struktur Data: Implementasi Algoritma Sorting Menggunakan Bahasa Java”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep algoritma pengurutan data (*sorting*) serta mengimplementasikannya menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu memahami cara kerja berbagai algoritma sorting, yaitu **Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort**, serta mengetahui perbedaan karakteristik dan efisiensi dari masing-masing algoritma.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai struktur data, khususnya penerapan algoritma sorting dalam pemrograman Java.

Padang, 5 Juni 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan data secara cepat dan efisien merupakan salah satu aspek yang sangat penting. Salah satu teknik yang sering digunakan dalam pengolahan data adalah **algoritma pengurutan (sorting)**. Sorting merupakan proses mengurutkan data berdasarkan kriteria tertentu, seperti urutan naik (*ascending*) atau urutan turun (*descending*). Data yang telah terurut akan mempermudah proses pencarian, pengelolaan, dan analisis data.

Terdapat berbagai algoritma sorting yang dapat digunakan, di antaranya **Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort**. Bubble Sort bekerja dengan membandingkan elemen yang berdekatan dan menukarnya jika urutannya tidak sesuai. Merge Sort menggunakan metode *divide and conquer* dengan membagi data menjadi beberapa bagian kecil, kemudian menggabungkannya kembali dalam keadaan terurut. Quick Sort juga menggunakan metode *divide and conquer* dengan memilih sebuah *pivot* sebagai acuan untuk membagi data. Sementara itu, Shell Sort merupakan pengembangan dari Insertion Sort yang melakukan pengurutan dengan jarak (*gap*) tertentu sehingga proses pengurutan menjadi lebih efisien.

Setiap algoritma memiliki karakteristik, kelebihan, dan tingkat efisiensi yang berbeda. Oleh karena itu, pemahaman terhadap berbagai metode sorting sangat diperlukan agar dapat memilih algoritma yang sesuai dengan kebutuhan. Selain itu, implementasi algoritma sorting menggunakan bahasa pemrograman Java dapat membantu mahasiswa memahami konsep algoritma secara lebih mendalam, baik melalui aplikasi berbasis teks maupun aplikasi berbasis **Graphical User Interface (GUI)**.

Pada praktikum ini dilakukan implementasi algoritma Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort menggunakan bahasa pemrograman Java. Praktikum ini bertujuan untuk meningkatkan pemahaman mahasiswa mengenai konsep dasar sorting serta kemampuan dalam mengimplementasikan algoritma pengurutan data secara efisien.

1.2 Tujuan

1. Memahami konsep dasar algoritma sorting.
2. Memahami cara kerja Bubble Sort GUI, Merge Sort, Quick Sort dan Shell Sort
3. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
4. Membandingkan metode dan efisiensi dari masing-masing algoritma sorting.
5. Melatih kemampuan pemrograman dalam mengelola dan mengurutkan data secara efisien. Melatih kemampuan pemrograman dalam mengelola data secara efisien menggunakan struktur data.

1.3 Manfaat

1. Menambah pemahaman tentang algoritma pengurutan data (sorting).
2. Membantu memahami perbedaan cara kerja Bubble Sort GUI, Merge Sort, Quick Sort, dan Shell Sort.
3. Melatih kemampuan implementasi algoritma dalam bahasa pemrograman Java.
4. Memberikan pengalaman dalam membuat aplikasi berbasis GUI.
5. Menjadi dasar pembelajaran algoritma dan struktur data yang lebih kompleks.

BAB II

PEMBAHASAN

Dalam pemrograman, algoritma merupakan langkah-langkah sistematis yang digunakan untuk menyelesaikan suatu masalah. Salah satu algoritma yang sering digunakan dalam pengolahan data adalah algoritma pengurutan (sorting). Sorting digunakan untuk menyusun data ke dalam urutan tertentu, seperti urutan menaik (ascending) atau menurun (descending), sehingga data lebih mudah diproses dan dianalisis.

Pada praktikum ini digunakan beberapa algoritma sorting, yaitu Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort. Keempat algoritma tersebut memiliki karakteristik dan cara kerja yang berbeda dalam melakukan pengurutan data. Implementasi algoritma sorting dilakukan menggunakan bahasa pemrograman Java, baik dalam bentuk program berbasis console maupun Graphical User Interface (GUI).

Dalam bahasa pemrograman Java, implementasi algoritma sorting dilakukan menggunakan array sebagai media penyimpanan data. Array digunakan untuk menyimpan sekumpulan nilai yang nantinya akan diurutkan menggunakan algoritma tertentu. Setiap algoritma memiliki langkah kerja yang berbeda dalam melakukan proses pengurutan data.

1. Merge Sort

Merge Sort merupakan algoritma pengurutan yang menggunakan metode Divide and Conquer, yaitu membagi data menjadi beberapa bagian yang lebih kecil, kemudian mengurutkannya dan menggabungkannya kembali menjadi satu bagian yang terurut. Algoritma ini bekerja secara rekursif hingga seluruh data tersusun dengan benar.

Langkah kerja Merge Sort:

1. Membagi array menjadi dua bagian yang lebih kecil.
2. Mengulangi proses pembagian hingga setiap bagian hanya memiliki satu elemen.
3. Menggabungkan kembali bagian-bagian tersebut dalam urutan yang benar.
4. Mengulangi proses penggabungan hingga seluruh data menjadi terurut.

Kelebihan Merge Sort adalah memiliki performa yang stabil dan efisien untuk data berukuran besar. Namun, algoritma ini membutuhkan memori tambahan untuk proses penggabungan data.

2. Quick Sort

Quick Sort merupakan algoritma pengurutan yang menggunakan metode Divide and Conquer dengan memilih sebuah elemen sebagai pivot. Data kemudian dibagi menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot.

Langkah kerja Quick Sort:

1. Memilih satu elemen sebagai pivot.
2. Membandingkan elemen lain dengan pivot.
3. Menempatkan elemen yang lebih kecil di sebelah kiri pivot dan yang lebih besar di sebelah kanan.
4. Mengulangi proses pada masing-masing bagian hingga seluruh data terurut.

Kelebihan Quick Sort adalah proses pengurutan yang cepat dan efisien. Namun, performanya dapat menurun apabila pemilihan pivot kurang tepat.

3. Shell Sort

Shell Sort merupakan pengembangan dari Insertion Sort yang menggunakan jarak tertentu (gap) dalam proses pengurutan. Dengan membandingkan elemen yang berjauhan terlebih dahulu, proses pengurutan menjadi lebih cepat.

Langkah kerja Shell Sort:

1. Menentukan nilai gap awal.
2. Membandingkan dan mengurutkan elemen yang berjarak sesuai gap.
3. Mengurangi nilai gap secara bertahap.
4. Mengulangi proses hingga gap bernilai satu dan seluruh data terurut.

Kelebihan Shell Sort adalah lebih cepat dibandingkan algoritma sorting sederhana seperti Bubble Sort dan Insertion Sort pada data berukuran sedang. Namun, efisiensinya dipengaruhi oleh pemilihan nilai gap.

4. GUI Bubble Sort

Graphical User Interface (GUI) merupakan tampilan antarmuka visual yang memungkinkan pengguna berinteraksi dengan program secara lebih mudah. Pada praktikum ini, algoritma Bubble Sort diimplementasikan dalam bentuk GUI menggunakan Java Swing.

GUI Bubble Sort dibuat agar pengguna dapat memasukkan data, menjalankan proses sorting, dan melihat hasil pengurutan secara langsung melalui tampilan visual. Komponen GUI yang digunakan antara lain:

1. JFrame sebagai jendela utama program.

2. JTextField untuk memasukkan data.
3. JButton untuk menjalankan proses Bubble Sort.
4. JTextArea untuk menampilkan hasil pengurutan.

Dengan adanya GUI, proses pengurutan data menggunakan Bubble Sort menjadi lebih interaktif dan mudah dipahami dibandingkan hanya menggunakan tampilan console. Selain itu, implementasi GUI juga melatih kemampuan mahasiswa dalam mengembangkan aplikasi desktop menggunakan bahasa pemrograman Java.

Melalui praktikum ini, mahasiswa dapat memahami konsep dasar algoritma sorting, mengetahui perbedaan cara kerja Merge Sort, Quick Sort, Shell Sort, dan Bubble Sort, serta mampu mengimplementasikannya ke dalam program Java baik berbasis console maupun GUI.

1. Langkah Kerja

1.1. Merge Sort

- Kode Program

```
1 package pekan0_2511531014;  
2  
3 public class MergeSort_2511531014 {  
4  
5     void Merge_2511531014(int arr_1014[], int l_1014, int m_1014, int r_1014) {  
6  
7         int n1_1014 = m_1014 - l_1014 + 1;  
8         int n2_1014 = r_1014 - m_1014;  
9  
10        int L_1014[] = new int[n1_1014];  
11        int R_1014[] = new int[n2_1014];  
12  
13        for (int i_1014 = 0; i_1014 < n1_1014; ++i_1014)  
14            L_1014[i_1014] = arr_1014[l_1014 + i_1014];  
15  
16        for (int j_1014 = 0; j_1014 < n2_1014; ++j_1014)  
17            R_1014[j_1014] = arr_1014[m_1014 + 1 + j_1014];  
18  
19        int i_1014 = 0;  
20        int j_1014 = 0;  
21        int k_1014 = l_1014;  
22  
23        while (i_1014 < n1_1014 && j_1014 < n2_1014) {  
24            if (L_1014[i_1014] <= R_1014[j_1014]) {  
25                arr_1014[k_1014] = L_1014[i_1014];  
26                i_1014++;  
27            } else {  
28                arr_1014[k_1014] = R_1014[j_1014];  
29                j_1014++;  
30            }  
31            k_1014++;  
32        }  
33  
34        while (i_1014 < n1_1014) {  
35            arr_1014[k_1014] = L_1014[i_1014];  
36            i_1014++;  
37            k_1014++;  
38        }  
39  
40        while (j_1014 < n2_1014) {  
41            arr_1014[k_1014] = R_1014[j_1014];  
42            j_1014++;  
43            k_1014++;  
44        }  
45    }  
46 }
```

```

1  void sort_2511531014(int arr_1014[], int l_1014, int r_1014) {
2      if (l_1014 < r_1014) {
3          int m_1014 = (l_1014 + r_1014) / 2;
4          sort_2511531014(arr_1014, l_1014, m_1014);
5          sort_2511531014(arr_1014, m_1014 + 1, r_1014);
6          Merge_2511531014(arr_1014, l_1014, m_1014, r_1014);
7      }
8  }
9
10 static void printArray_2511531014(int arr_1014[]) {
11     int n_1014 = arr_1014.length;
12     for (int i_1014 = 0; i_1014 < n_1014; ++i_1014)
13         System.out.print(arr_1014[i_1014] + " ");
14     System.out.println();
15 }
16
17 public static void main(String args[]) {
18     int arr_1014[] = {12, 11, 13, 5, 6, 7};
19     System.out.println("Sebelum terurut");
20     printArray_2511531014(arr_1014);
21
22     MergeSort_2511531014 ob_1014 = new MergeSort_2511531014();
23     ob_1014.sort_2511531014(arr_1014, 0, arr_1014.length - 1);
24     System.out.println("\nSesudah Terurut menggunakan Merge Sort");
25     printArray_2511531014(arr_1014);
26 }

```

- Pembahasan

Program ini dibuat dengan mendefinisikan class MergeSort_2511531014 yang digunakan untuk mengimplementasikan algoritma **Merge Sort** pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (*ascending*) menggunakan metode *Divide and Conquer*.

1. Class ini memiliki method Merge_2511531014(int arr_1014[], int l_1014, int m_1014, int r_1014) yang digunakan untuk menggabungkan dua bagian array yang telah terurut menjadi satu array yang tetap terurut. Method ini menerima parameter berupa array, indeks kiri, indeks tengah, dan indeks kanan.
2. Pada method Merge_2511531014(), variabel n1_1014 dan n2_1014 digunakan untuk menentukan jumlah elemen pada sub-array kiri dan sub-array kanan. Kemudian dibuat array sementara L_1014[] dan R_1014[] untuk menyimpan data dari kedua bagian array yang akan digabungkan.
3. Proses penyalinan data ke array sementara dilakukan menggunakan perulangan for. Array L_1014[] digunakan untuk menyimpan elemen dari bagian kiri, sedangkan array R_1014[] digunakan untuk menyimpan elemen dari bagian kanan.
4. Variabel i_1014, j_1014, dan k_1014 digunakan sebagai indeks penelusuran. Variabel i_1014 digunakan untuk menelusuri array kiri, j_1014 untuk menelusuri array kanan, dan k_1014 untuk menentukan posisi penyimpanan hasil penggabungan pada array utama.
5. Proses penggabungan dilakukan menggunakan perulangan while. Program akan membandingkan elemen pada array kiri dan array kanan.

Nilai yang lebih kecil akan ditempatkan ke array utama terlebih dahulu sehingga urutan data tetap terjaga secara ascending.

6. Setelah salah satu array habis diproses, perulangan while berikutnya digunakan untuk menyalin sisa elemen yang masih terdapat pada array kiri atau array kanan ke dalam array utama sampai seluruh elemen selesai digabungkan.

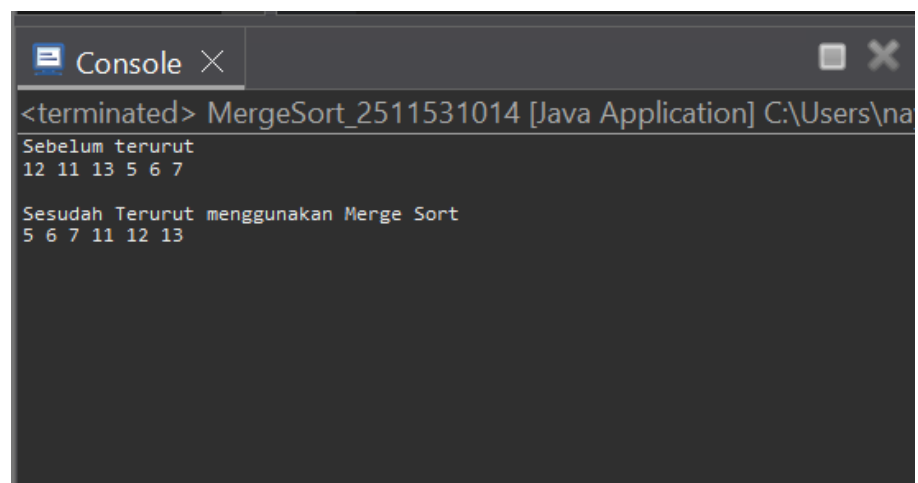
7. Class ini juga memiliki method `sort_2511531014()` yang berfungsi untuk membagi array menjadi beberapa bagian yang lebih kecil secara rekursif. Method ini akan terus membagi array hingga setiap bagian hanya berisi satu elemen, kemudian memanggil method `Merge_2511531014()` untuk menggabungkannya kembali secara terurut.

8. Method `printArray_2511531014()` digunakan untuk menampilkan isi array sebelum dan sesudah proses pengurutan sehingga hasil Merge Sort dapat diamati dengan jelas.

9. Pada method `main()`, program membuat sebuah array integer yang berisi data yang belum terurut. Data tersebut kemudian ditampilkan, diurutkan menggunakan method `sort_2511531014()`, dan hasil pengurutan ditampilkan kembali menggunakan method `printArray_2511531014()`.

10. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma **Merge Sort** dalam mengurutkan data integer menggunakan metode *Divide and Conquer*, yaitu membagi array menjadi bagian-bagian kecil, mengurutkannya, lalu menggabungkannya kembali hingga seluruh data tersusun secara ascending.

OUTPUT



```
<terminated> MergeSort_2511531014 [Java Application] C:\Users\na
Sebelum terurut
12 11 13 5 6 7

Setelah Terurut menggunakan Merge Sort
5 6 7 11 12 13
```

1.2. Quick Sort

- Kode Program

```
package penang_2511531014;
2
3 public class QuickSort_2511531014 {
4     static void swap(int[] arr_1014, int i_1014, int j_1014)
5     {
6         int temp_1014 = arr_1014[i_1014];
7         arr_1014[i_1014] = arr_1014[j_1014];
8         arr_1014[j_1014] = temp_1014;
9     }
10    //Metode tambahan untuk mengatur pivot menggunakan median-of-Three
11    static void medianOfThree(int[] arr_1014, int low_1014, int high_1014)
12    {
13        int mid_1014 = low_1014 + (high_1014 - low_1014) / 2;
14
15        //Memastikan elemen low, mid, dan high
16        if (arr_1014[low_1014] > arr_1014[mid_1014]) {
17            swap(arr_1014, low_1014, mid_1014);
18        }
19        if (arr_1014[low_1014] > arr_1014[high_1014]) {
20            swap(arr_1014, low_1014, high_1014);
21        }
22        if (arr_1014[mid_1014] > arr_1014[high_1014]) {
23            swap(arr_1014, mid_1014, high_1014);
24        }
25        swap(arr_1014, mid_1014, high_1014);
26    }
27    static int partition_2511531014(int[] arr_1014, int low_1014, int high_1014)
28    {
29        //Panggil fungsi medianOfThree sebagai pengganti pivot
30        medianOfThree(arr_1014, low_1014, high_1014);
31
32        int pivot_1014 = arr_1014[high_1014]; //sediakan arr[high] sudah berisi nilai median
33        int i_1014 = (low_1014 - 1);
34
35        for (int j_1014 = low_1014; j_1014 <= high_1014 - 1; j_1014++) {
36            //jika elemen saat ini lebih kecil dari pivot maka pindah ke posisi sebelum pivot
37            if (arr_1014[j_1014] < pivot_1014) {
38                //Increment indeks elemen yang lebih kecil
39                i_1014++;
40                swap(arr_1014, i_1014, j_1014);
41            }
42        }
43        swap(arr_1014, i_1014 + 1, high_1014);
44        return (i_1014 + 1);
45    }
46
47    static void quickSort_2511531014(int[] arr_1014, int low_1014, int high_1014)
48    {
49        if (low_1014 < high_1014) {
50            int pi_1014 = partition_2511531014(arr_1014, low_1014, high_1014);
51            quickSort_2511531014(arr_1014, low_1014, pi_1014 - 1);
52            quickSort_2511531014(arr_1014, pi_1014 + 1, high_1014);
53        }
54    }
55
56    public static void printArr_2511531014(int[] arr_1014)
57    {
58        for (int i_1014 = 0; i_1014 < arr_1014.length; i_1014++) {
59            System.out.print(arr_1014[i_1014] + " ");
60        }
61        System.out.println();
62    }
63
64 }
```

- Pembahasan

Program ini dibuat dengan mendefinisikan class QuickSort_2511531014 yang digunakan untuk mengimplementasikan algoritma **Quick Sort** pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (*ascending*) menggunakan metode *Divide and Conquer*.

1. Class ini memiliki method swap() yang digunakan untuk menukar posisi dua elemen pada array. Method ini membantu proses pengurutan ketika terdapat elemen yang harus dipindahkan ke posisi yang sesuai.
2. Method medianOfThree() digunakan untuk menentukan nilai pivot menggunakan teknik **Median of Three**, yaitu dengan membandingkan elemen pertama (low_1014), elemen tengah (mid_1014), dan elemen terakhir (high_1014). Teknik ini digunakan agar pemilihan pivot lebih optimal sehingga performa Quick Sort menjadi lebih baik.
3. Setelah nilai median diperoleh, elemen median dipindahkan ke posisi terakhir array menggunakan method swap(). Nilai tersebut kemudian digunakan sebagai pivot pada proses partisi.
4. Method partition_2511531014() digunakan untuk membagi array menjadi dua bagian berdasarkan nilai pivot. Elemen yang lebih kecil

dari pivot akan ditempatkan di sebelah kiri, sedangkan elemen yang lebih besar akan ditempatkan di sebelah kanan.

5. Pada method `partition_2511531014()`, variabel `i_1014` digunakan untuk menandai posisi elemen yang lebih kecil dari pivot, sedangkan variabel `j_1014` digunakan untuk menelusuri seluruh elemen array yang akan dibandingkan dengan pivot.

6. Jika ditemukan elemen yang nilainya lebih kecil dari pivot, maka indeks `i_1014` akan bertambah dan elemen tersebut ditukar menggunakan method `swap()`. Setelah seluruh elemen diperiksa, pivot ditempatkan pada posisi yang benar di dalam array.

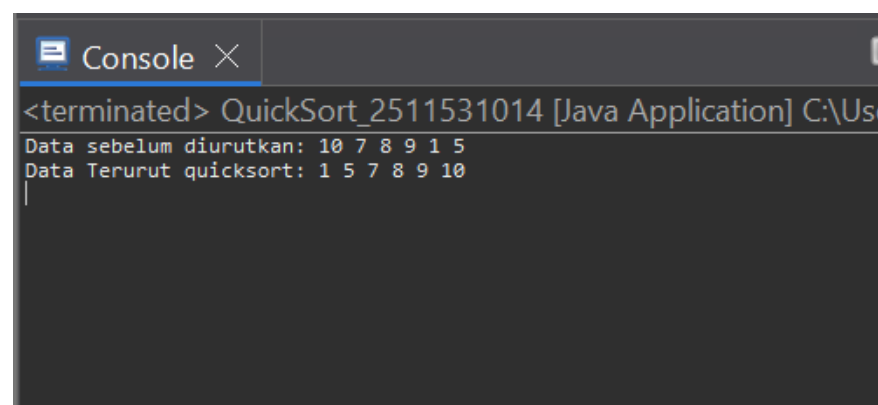
7. Method `quickSort_2511531014()` digunakan untuk menjalankan proses pengurutan secara rekursif. Setelah mendapatkan posisi pivot dari method `partition_2511531014()`, program akan mengurutkan bagian kiri dan bagian kanan array hingga seluruh data terurut.

8. Method `printArr_2511531014()` digunakan untuk menampilkan isi array sebelum dan sesudah proses pengurutan sehingga hasil Quick Sort dapat dilihat dengan jelas.

9. Pada method `main()`, program membuat sebuah array integer yang berisi data yang belum terurut. Data tersebut kemudian ditampilkan, diurutkan menggunakan method `quickSort_2511531014()`, dan hasil pengurutan ditampilkan kembali menggunakan method `printArr_2511531014()`.

10. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma **Quick Sort** dengan teknik **Median of Three Pivot**, yaitu memilih nilai tengah dari tiga elemen sebagai pivot, kemudian membagi array menjadi beberapa bagian dan mengurutkannya secara rekursif hingga seluruh data tersusun secara ascending.

OUTPUT



```
<terminated> QuickSort_2511531014 [Java Application] C:\Us
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

1.3. Shell Sort

- Kode Program

```
package Belajar_2511531014;

public class QuickSort_2511531014 {
    static void swap(int[] arr_1014, int i_1014, int j_1014)
    {
        int temp_1014 = arr_1014[i_1014];
        arr_1014[i_1014] = arr_1014[j_1014];
        arr_1014[j_1014] = temp_1014;
    }

    //Metode tambahan untuk memilih pivot menggunakan median-of-Three
    static void medianOfThree(int[] arr_1014, int low_1014, int high_1014)
    {
        int mid_1014 = low_1014 + (high_1014 - low_1014) / 2;

        //Urutkan elemen low, mid, dan high
        if (arr_1014[low_1014] > arr_1014[mid_1014]) {
            swap(arr_1014, low_1014, mid_1014);
        }
        if (arr_1014[low_1014] > arr_1014[high_1014]) {
            swap(arr_1014, low_1014, high_1014);
        }
        if (arr_1014[mid_1014] > arr_1014[high_1014]) {
            swap(arr_1014, mid_1014, high_1014);
        }
        swap(arr_1014, mid_1014, high_1014);
    }

    static int partition_2511531014 (int[] arr_1014, int low_1014, int high_1014)
    {
        //Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree(arr_1014, low_1014, high_1014);

        int pivot_1014 = arr_1014[high_1014]; //sekarang arr[high] sudah berisi nilai median
        int i_1014 = (low_1014 - 1);

        for (int j_1014 = low_1014; j_1014 <= high_1014 - 1; j_1014++) {
            //Jika elemen saat ini lebih kecil dari atau sama dengan pivot
            if (arr_1014[j_1014] < pivot_1014) {
                //Increment indeks elemen yang lebih kecil
                i_1014++;
                swap(arr_1014, i_1014, j_1014);
            }
        }
        swap(arr_1014, i_1014 + 1, high_1014);
        return (i_1014 + 1);
    }

    static void quickSort_2511531014(int[] arr_1014, int low_1014, int high_1014)
    {
        if (low_1014 < high_1014) {
            int pi_1014 = partition_2511531014(arr_1014, low_1014, high_1014);
            quickSort_2511531014(arr_1014, low_1014, pi_1014 - 1);
            quickSort_2511531014(arr_1014, pi_1014 + 1, high_1014);
        }
    }

    public static void printArr_2511531014(int[] arr_1014)
    {
        for (int i_1014 = 0; i_1014 < arr_1014.length; i_1014++) {
            System.out.print(arr_1014[i_1014] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args)
    {
        int[] arr_1014 = {10, 7, 8, 9, 1, 5};
        int N_1014 = arr_1014.length;

        System.out.print("Data sebelum diurutkan: ");
        printArr_2511531014(arr_1014);

        quickSort_2511531014(arr_1014, 0, N_1014 - 1);

        System.out.print("Data Terurut quicksort: ");
        printArr_2511531014(arr_1014);
    }
}
```

- Pembahasan

Program ini dibuat dengan mendefinisikan class QuickSort_2511531014 yang digunakan untuk mengimplementasikan algoritma **Quick Sort** pada array bertipe integer. Class ini berfungsi untuk melakukan proses pengurutan data dari nilai terkecil ke terbesar (*ascending*) menggunakan metode *Divide and Conquer* dengan teknik **Median of Three Pivot**.

1. Class ini memiliki method swap() yang digunakan untuk menukar posisi dua elemen pada array. Method ini membantu proses pengurutan ketika terdapat elemen yang harus dipindahkan ke posisi yang sesuai.

2. Method medianOfThree() digunakan untuk menentukan nilai pivot dengan membandingkan tiga elemen, yaitu elemen pertama (low_1014), elemen tengah (mid_1014), dan elemen terakhir (high_1014). Setelah dibandingkan, nilai median dipindahkan ke posisi terakhir array dan digunakan sebagai pivot.

3. Penggunaan teknik *Median of Three* bertujuan untuk memperoleh pivot yang lebih baik dibandingkan memilih elemen pertama atau terakhir secara langsung. Dengan demikian, kemungkinan terjadinya pembagian array yang tidak seimbang dapat dikurangi.

4. Method `partition_2511531014()` digunakan untuk membagi array menjadi dua bagian berdasarkan nilai pivot. Semua elemen yang lebih kecil dari pivot akan ditempatkan di sebelah kiri, sedangkan elemen yang lebih besar akan ditempatkan di sebelah kanan.

5. Pada method `partition_2511531014()`, variabel `pivot_1014` digunakan untuk menyimpan nilai pivot, variabel `i_1014` digunakan sebagai penanda posisi elemen yang lebih kecil dari pivot, dan variabel `j_1014` digunakan untuk menelusuri seluruh elemen array yang akan dibandingkan.

6. Selama proses perulangan `for`, setiap elemen dibandingkan dengan pivot. Jika nilai elemen lebih kecil dari pivot, maka indeks `i_1014` akan bertambah dan posisi elemen ditukar menggunakan method `swap()`. Setelah seluruh elemen diperiksa, pivot ditempatkan pada posisi yang benar sehingga menghasilkan dua bagian array yang siap diurutkan.

7. Method `quickSort_2511531014()` digunakan untuk menjalankan proses pengurutan secara rekursif. Setelah mendapatkan posisi pivot dari method `partition_2511531014()`, program akan memanggil dirinya sendiri untuk mengurutkan bagian kiri dan bagian kanan array hingga seluruh data terurut.

8. Method `printArr_2511531014()` digunakan untuk menampilkan isi array sebelum dan sesudah proses pengurutan sehingga hasil Quick Sort dapat diamati dengan jelas.

9. Pada method `main()`, program membuat sebuah array integer yang berisi data yang belum terurut. Data tersebut kemudian ditampilkan, diurutkan menggunakan method `quickSort_2511531014()`, dan hasil pengurutan ditampilkan kembali untuk menunjukkan perubahan data setelah proses sorting.

10. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma **Quick Sort** dengan teknik **Median of Three Pivot**, yaitu memilih nilai median dari tiga elemen sebagai pivot, membagi array menjadi dua bagian, kemudian mengurutkannya secara rekursif hingga seluruh data tersusun secara ascending.

OUPUT

```
Console X
<terminated> ShellSort_2511531014 [Java Application] C
sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort) : 1 2 3 4 5 6 7 8 9 10
```

1.4. Bubble Sort GUI

- Kode Program

```
1 package pekan0_2511531014;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9
10 import javax.swing.BorderFactory;
11 import javax.swing.JButton;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JOptionPane;
15 import javax.swing.JPanel;
16 import javax.swing.JScrollPane;
17 import javax.swing.JTextArea;
18 import javax.swing.JTextField;
19 import javax.swing.SwingConstants;
20 import javax.swing.border.EmptyBorder;
21
22 public class BubbleSortGUI_2511531014 extends JFrame {
23
24     private static final long serialVersionUID = 1L;
25
26     private JPanel contentPane;
27     private JTextField inputField;
28     private JButton inputButton;
29     private JButton stepButton;
30     private JButton resetButton;
31     private JPanel panelArray;
32     private JTextArea stepArea;
33
34     private int[] array;
35     private JLabel[] labelArray;
36
37     private int i = 0;
38     private int j = 0;
39     private int stepCount = 1;
40     private boolean sorting = false;
```

```

41
42 public static void main(String[] args) {
43
44     EventQueue.invokeLater(new Runnable() {
45
46         public void run() {
47
48             try {
49
50                 BubbleSortGUI_2511531014 frame_1014 =
51                     new BubbleSortGUI_2511531014();
52
53                 frame_1014.setVisible(true);
54
55             } catch (Exception e_1014) {
56
57                 e_1014.printStackTrace();
58             }
59         }
60     });
61 }
62
63 public BubbleSortGUI_2511531014() {
64
65     setTitle("Visualisasi Bubble Sort");
66     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
67     setBounds(100, 100, 900, 500);
68
69     contentPane = new JPanel();
70     contentPane.setBorder(new EmptyBorder(10, 10, 10, 10));
71     contentPane.setLayout(new BorderLayout(10, 10));
72     setContentPane(contentPane);
73
74     JPanel topPanel_1014 = new JPanel(new FlowLayout());
75
76     JLabel lblInput_1014 =
77         new JLabel("Masukkan angka (pisahkan koma):");
78
79     inputField = new JTextField(25);
80
81     inputButton = new JButton("Tampilkan");
82
83     topPanel_1014.add(lblInput_1014);
84     topPanel_1014.add(inputField);
85     topPanel_1014.add(inputButton);
86
87     contentPane.add(topPanel_1014, BorderLayout.NORTH);
88

```

```

9     panelArray = new JPanel();
10     panelArray.setLayout(new FlowLayout());
11
12     contentPane.add(panelArray, BorderLayout.CENTER);
13
14     stepArea = new JTextArea();
15     stepArea.setEditable(false);
16
17     JScrollPane scrollPane_1014 =
18         new JScrollPane(stepArea);
19
20     scrollPane_1014.setPreferredSize(
21         new Dimension(300, 0));
22
23     contentPane.add(scrollPane_1014,
24         BorderLayout.EAST);
25
26     JPanel bottomPanel_1014 = new JPanel();
27
28     stepButton =
29         new JButton("Sorting Selanjutnya");
30
31     resetButton =
32         new JButton("Reset");
33
34     stepButton.setEnabled(false);
35
36     bottomPanel_1014.add(stepButton);
37     bottomPanel_1014.add(resetButton);
38
39     contentPane.add(bottomPanel_1014,
40         BorderLayout.SOUTH);
41
42     inputButton.addActionListener(
43         e_1014 -> setArrayFromInput_2511531014());
44
45     stepButton.addActionListener(
46         e_1014 -> nextStep_2511531014());
47
48     resetButton.addActionListener(
49         e_1014 -> reset_2511531014());
50 }
51

```

```

181
182 private void setArrayFromInput_2511531014() {
183
184     String text_1014 =
185         inputField.getText().trim();
186
187     if (text_1014.isEmpty())
188         return;
189
190     String[] parts_1014 =
191         text_1014.split(",");
192
193     array = new int[parts_1014.length];
194
195     try {
196
197         for (int k_1014 = 0;
198             k_1014 < parts_1014.length;
199             k_1014++) {
200
201             array[k_1014] =
202                 Integer.parseInt(
203                     parts_1014[k_1014]
204                         .trim());
205
206         }
207
208     } catch (NumberFormatException e_1014) {
209
210         JOptionPane.showMessageDialog(
211             this,
212             "Masukkan hanya angka yang dipisahkan koma!",
213             "Error",
214             JOptionPane.ERROR_MESSAGE);
215
216         return;
217     }
218
219     i = 0;
220     j = 0;
221     stepCount = 1;
222     sorting = true;
223
224     stepButton.setEnabled(true);
225
226     stepArea.setText("");
227
228     panelArray.removeAll();
229
230     labelArray =
231         new JLabel[array.length];

```

```

181
182     for (int k_1014 = 0;
183         k_1014 < array.length;
184         k_1014++) {
185
186         labelArray[k_1014] =
187             new JLabel(
188                 String.valueOf(
189                     array[k_1014]));
190
191         labelArray[k_1014].setFont(
192             new Font(
193                 "Arial",
194                 Font.BOLD,
195                 24));
196
197         labelArray[k_1014].setOpaque(true);
198
199         labelArray[k_1014].setBackground(
200             Color.WHITE);
201
202         labelArray[k_1014].setBorder(
203             BorderFactory
204                 .createLineBorder(
205                     Color.BLACK));
206
207         labelArray[k_1014].setPreferredSize(
208             new Dimension(60, 50));
209
210         labelArray[k_1014]
211             .setHorizontalAlignment(
212                 SwingConstants.CENTER);
213
214         panelArray.add(
215             labelArray[k_1014]);
216     }
217
218     panelArray.revalidate();
219     panelArray.repaint();
220 }
221
222 private void nextStep_2511531014() {
223
224     if (!sorting)
225         return;
226
227     resetColors_2511531014();
228
229     if (j < array.length - i - 1) {
230
231         labelArray[i]

```



```

231         labelArray[j]
232             .setBackground(Color.YELLOW);
233
234         labelArray[j + 1]
235             .setBackground(Color.YELLOW);
236
237         int nilaiKiri_1014 =
238             array[j];
239
240         int nilaiKanan_1014 =
241             array[j + 1];
242
243         if (array[j] > array[j + 1]) {
244
245             int temp_1014 =
246                 array[j];
247
248             array[j] =
249                 array[j + 1];
250
251             array[j + 1] =
252                 temp_1014;
253
254             updateLabels_2511531014();
255
256             labelArray[j]
257                 .setBackground(Color.RED);
258
259             labelArray[j + 1]
260                 .setBackground(Color.RED);
261
262             stepArea.append(
263                 "Langkah "
264                     + stepCount
265                     + " : Tukar "
266                     + nilaiKiri_1014
267                     + " dengan "
268                     + nilaiKanan_1014
269                     + "\n");
270
271         } else {
272
273             stepArea.append(
274                 "Langkah "
275                     + stepCount
276                     + " : Tidak ada pertukaran ("
277                     + nilaiKiri_1014
278                     + " dan "
279                     + nilaiKanan_1014
280                     + ") \n");

```

```

280                     + ") \n");
281
282         }
283
284         j++;
285     } else {
286
287         labelArray[
288             array.length - i - 1]
289             .setBackground(Color.GREEN);
290
291         j = 0;
292         i++;
293     }
294
295     stepCount++;
296
297     if (i >= array.length - 1) {
298
299         sorting = false;
300
301         for (JLabel lbl_1014
302             : labelArray) {
303
304             lbl_1014.setBackground(
305                 Color.GREEN);
306         }
307
308         stepButton.setEnabled(false);
309
310         stepArea.append(
311             "\n=== SORTING SELESAI ===\n");
312
313         JOptionPane.showMessageDialog(
314             this,
315             "Sorting selesai!");
316     }
317
318     private void updateLabels_2511531014() {
319
320         for (int k_1014 = 0;
321             k_1014 < array.length;
322             k_1014++) {
323
324             labelArray[k_1014]
325                 .setText(
326                     String.valueOf(
327                         array[k_1014]));
328

```

```

29     }
30 }
31
32 private void resetColors_2511531014() {
33     if (labelArray == null)
34         return;
35
36     for (JLabel lbl_1014
37          : labelArray) {
38         if (!Color.GREEN.equals(
39             lbl_1014.getBackground())) {
40             lbl_1014.setBackground(
41                 Color.WHITE);
42         }
43     }
44 }
45
46 private void reset_2511531014() {
47     inputField.setText("");
48     panelArray.removeAll();
49     panelArray.revalidate();
50     panelArray.repaint();
51     stepArea.setText("");
52     stepButton.setEnabled(false);
53
54     array = null;
55     labelArray = null;
56     sorting = false;
57
58     i = 0;
59     j = 0;
60     stepCount = 1;
61 }
62 }
63
64 }
65
66 }
67
68 }
69
70 }
71
72 }

```

- Pembahasan

Program ini dibuat dengan mendefinisikan class `BubbleSortGUI_2511531014` yang digunakan untuk mengimplementasikan algoritma **Bubble Sort** dalam bentuk **Graphical User Interface (GUI)** menggunakan Java Swing. Program ini memungkinkan pengguna memasukkan data, melihat proses pengurutan langkah demi langkah, serta menampilkan hasil pengurutan secara visual.

1. Class ini merupakan turunan dari `JFrame` yang berfungsi sebagai jendela utama aplikasi. Berbagai komponen GUI seperti `JTextField`, `JPanel`, `JLabel`, dan `JTextArea` digunakan untuk membangun tampilan program.
2. Method konstruktor `BubbleSortGUI_2511531014()` digunakan untuk mengatur tampilan antarmuka, seperti membuat panel input, panel visualisasi array, area penjelasan langkah sorting, serta tombol untuk menjalankan proses sorting dan reset data.
3. Method `setArrayFromInput_2511531014()` digunakan untuk membaca data yang dimasukkan pengguna melalui `JTextField`. Data yang dipisahkan dengan tanda koma akan diubah menjadi array integer dan ditampilkan dalam bentuk label pada panel visualisasi.
4. Pada proses visualisasi, setiap elemen array ditampilkan menggunakan `JLabel`. Label tersebut digunakan untuk menunjukkan

nilai data yang sedang diproses selama algoritma Bubble Sort berlangsung.

5. Method `nextStep_2511531014()` digunakan untuk menjalankan satu langkah proses Bubble Sort setiap kali tombol **Sorting Selanjutnya** ditekan. Program akan membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah.

6. Selama proses pengurutan, warna label digunakan untuk memperjelas visualisasi. Warna kuning menunjukkan elemen yang sedang dibandingkan, warna merah menunjukkan elemen yang ditukar, sedangkan warna hijau menunjukkan elemen yang sudah berada pada posisi yang benar.

7. Method `updateLabels_2511531014()` digunakan untuk memperbarui tampilan nilai pada label setelah terjadi pertukaran elemen. Dengan demikian pengguna dapat melihat perubahan data secara langsung pada layar.

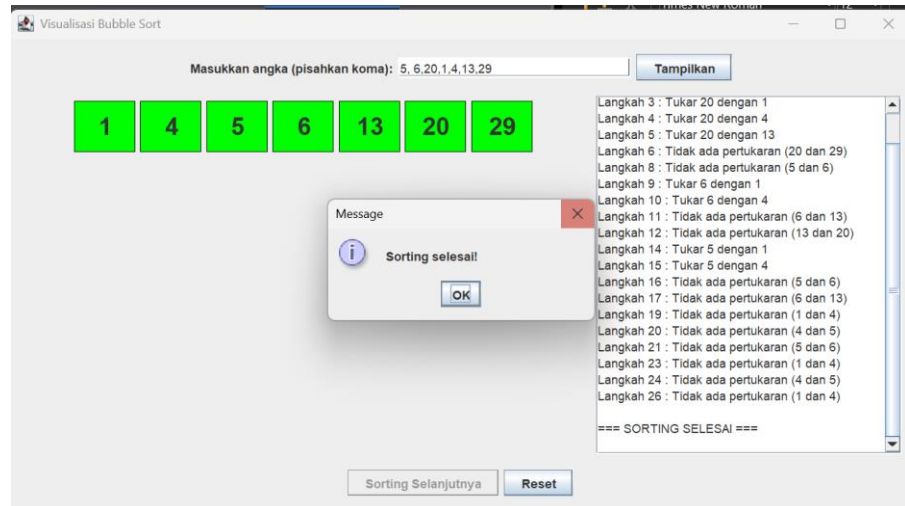
8. Method `resetColors_2511531014()` digunakan untuk mengembalikan warna label ke kondisi awal sebelum proses perbandingan berikutnya dilakukan, sehingga visualisasi menjadi lebih jelas dan mudah dipahami.

9. Method `reset_2511531014()` digunakan untuk menghapus seluruh data yang telah dimasukkan, membersihkan area visualisasi, mengosongkan riwayat langkah sorting, serta mengembalikan program ke kondisi awal.

10. Pada method `main()`, program dijalankan menggunakan `EventQueue.invokeLater()` untuk menampilkan jendela GUI. Pengguna kemudian dapat memasukkan data, menjalankan Bubble Sort secara bertahap, melihat setiap proses perbandingan dan pertukaran data, serta mengamati hasil pengurutan hingga selesai.

11. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma **Bubble Sort** secara visual. Melalui GUI, pengguna dapat melihat setiap langkah perbandingan dan pertukaran elemen secara langsung sehingga proses pengurutan data menjadi lebih interaktif dan mudah dipahami dibandingkan tampilan berbasis console.

OUTPUT



TUGAS PRAKTIKUM 8

Algoritma Sorting (Quick Sort)

Saya memilih algoritma **Quick Sort** karena algoritma ini memiliki proses pengurutan yang cepat, efisien, dan menggunakan konsep rekursi yang menarik untuk dipelajari. Selain itu, Quick Sort merupakan salah satu algoritma sorting yang banyak digunakan dalam pemrograman karena performanya yang baik. Dengan mengimplementasikan Quick Sort, saya dapat memahami cara kerja pivot, partition, dan rekursi dalam proses pengurutan data. Oleh karena itu, algoritma ini dipilih sebagai implementasi pada praktikum pengurutan playlist lagu

Class Lagu_2511531014

Kode Program

```

1 package TugasPekan8_2511531014;
2
3 public class Lagu_2511531014 {
4     String judul_1014;
5     String penyanyi_1014;
6     int durasi_1014;
7
8     public Lagu_2511531014(String judul_1014,
9                             String penyanyi_1014,
10                            int durasi_1014) {
11
12         this.judul_1014 = judul_1014;
13         this.penyanyi_1014 = penyanyi_1014;
14         this.durasi_1014 = durasi_1014;
15     }
16 }
17
18

```

Pembahasan

Program ini dibuat dengan mendefinisikan class **Lagu_2511531014** yang digunakan sebagai representasi data lagu pada program playlist. Class ini berfungsi untuk menyimpan informasi setiap lagu yang nantinya akan diurutkan menggunakan algoritma Quick Sort.

1. Class ini memiliki atribut **judul_1014** bertipe String untuk menyimpan judul lagu, atribut **penyanyi_1014** bertipe String untuk menyimpan nama penyanyi, dan atribut **durasi_1014** bertipe int untuk menyimpan lama durasi lagu dalam satuan detik.
2. Program menggunakan konstruktor **Lagu_2511531014(String judul_1014, String penyanyi_1014, int durasi_1014)** untuk menginisialisasi objek lagu. Konstruktor ini akan menerima data judul, penyanyi, dan durasi, kemudian menyimpannya ke dalam atribut class menggunakan keyword **this**.
3. Setiap objek yang dibuat dari class ini merepresentasikan satu data lagu yang akan disimpan ke dalam array **dataLagu_1014** pada class **Sorting_2511531014**.
4. Secara keseluruhan, class **Lagu_2511531014** berfungsi sebagai struktur data sederhana yang digunakan untuk menyimpan informasi lagu sebelum dilakukan proses pengurutan menggunakan algoritma Quick Sort berdasarkan durasi lagu secara ascending (dari durasi terpendek ke terpanjang).

Class **Sorting_2511531014**

Kode Program

```

1 package TugasPekan8_2511531014;
2
3
4 import java.util.Scanner;
5
6 public class Sorting_2511531014 {
7
8     Lagu_2511531014[] dataLagu_1014 =
9         new Lagu_2511531014[20];
10
11     int jumlahData_1014 = 0;
12
13     // input data lagu
14     public void inputData_1014() {
15
16         Scanner input_1014 = new Scanner(System.in);
17
18         System.out.print("Jumlah lagu (minimal 7): ");
19         jumlahData_1014 = input_1014.nextInt();
20         input_1014.nextLine();
21
22         for (int i_1014 = 0; i_1014 < jumlahData_1014; i_1014++) {
23
24             System.out.println("\nData Lagu ke-" + (i_1014 + 1));
25
26             System.out.print("Judul : ");
27             String judul_1014 = input_1014.nextLine();
28
29             System.out.print("Penyanyi : ");
30             String penyanyi_1014 = input_1014.nextLine();
31
32             System.out.print("Durasi (detik) : ");
33             int durasi_1014 = input_1014.nextInt();
34             input_1014.nextLine();
35
36             dataLagu_1014[i_1014] =
37                 new Lagu_2511531014(
38                     judul_1014,
39                     penyanyi_1014,
40                     durasi_1014);
41         }
42     }

```

```

44     // tukarkan data lagu
45     public void swap_1014(int i_1014, int j_1014) {
46
47         Lagu_2511531014 temp_1014 =
48             dataLagu_1014[i_1014];
49
50         dataLagu_1014[i_1014] =
51             dataLagu_1014[j_1014];
52
53         dataLagu_1014[j_1014] =
54             temp_1014;
55     }
56
57     // partition quick sort
58     public int partition_1014(int low_1014,
59                             int high_1014) {
60
61         int pivot_1014 =
62             dataLagu_1014[high_1014].durasi_1014;
63
64         int i_1014 = low_1014 - 1;
65
66         for (int j_1014 = low_1014;
67             j_1014 < high_1014;
68             j_1014++) {
69
70             if (dataLagu_1014[j_1014].durasi_1014
71                 < pivot_1014) {
72
73                 i_1014++;
74                 swap_1014(i_1014, j_1014);
75             }
76         }
77
78         swap_1014(i_1014 + 1, high_1014);
79
80         return i_1014 + 1;
81     }
82
83     // quick sort
84     public void quickSort_1014(int low_1014,
85                               int high_1014) {

```

```

84 public void quickSort_1014(int low_1014,
85                             int high_1014) {
86
87     if (low_1014 < high_1014) {
88
89         int pi_1014 =
90             partition_1014(
91                 low_1014,
92                 high_1014);
93
94         quickSort_1014(
95             low_1014,
96             pi_1014 - 1);
97
98         quickSort_1014(
99             pi_1014 + 1,
100             high_1014);
101     }
102 }
103
104 // tampil data
105 public void tampilData_1014() {
106
107     for (int i_1014 = 0;
108          i_1014 < jumlahData_1014;
109          i_1014++) {
110
111         System.out.println(
112             (i_1014 + 1) + ". "
113             + dataLagu_1014[i_1014].judul_1014
114             + " - "
115             + dataLagu_1014[i_1014].penyanyi_1014
116             + " - "
117             + dataLagu_1014[i_1014].durasi_1014
118             + " detik");
119     }
120 }
121

```

```

22 public static void main(String[] args) {
23
24     Sorting_2511531014 playlist_1014 =
25         new Sorting_2511531014();
26
27     System.out.println(
28         "=== Sorting Playlist NIM: 2511531014 ===");
29
30     playlist_1014.inputData_1014();
31
32     System.out.println(
33         "\n=== Data Sebelum Sorting ===");
34
35     playlist_1014.tampilData_1014();
36
37     playlist_1014.quickSort_1014(
38         0,
39         playlist_1014.jumlahData_1014 - 1);
40
41     System.out.println(
42         "\n=== Data Setelah Quick Sort (Durasi Asc) ===");
43
44     playlist_1014.tampilData_1014();
45 }
46 }

```

Pembahasan

Sorting_2511531014 yang digunakan untuk mengelola data playlist lagu serta mengimplementasikan algoritma **Quick Sort** untuk mengurutkan lagu berdasarkan durasi dari yang terpendek hingga terpanjang.

1. Class ini memiliki atribut **dataLagu_1014** berupa array objek Lagu_2511531014 dengan kapasitas 20 data yang digunakan untuk menyimpan informasi lagu. Selain itu terdapat atribut **jumlahData_1014** yang digunakan untuk menyimpan jumlah lagu yang dimasukkan oleh pengguna.
2. Method **inputData_1014()** digunakan untuk memasukkan data lagu melalui keyboard menggunakan class Scanner. Pada method ini

pengguna diminta memasukkan judul lagu, nama penyanyi, dan durasi lagu, kemudian data tersebut disimpan ke dalam array `dataLagu_1014`.

3. Method **`swap_1014()`** digunakan untuk menukar posisi dua objek lagu dalam array. Method ini berperan penting dalam proses pengurutan karena digunakan saat terjadi pertukaran data selama algoritma Quick Sort berjalan.

4. Method **`partition_1014()`** digunakan untuk membagi data berdasarkan nilai pivot. Pada program ini pivot diambil dari durasi lagu pada elemen terakhir array. Selanjutnya setiap data dibandingkan dengan pivot untuk menentukan apakah harus ditempatkan di sebelah kiri atau kanan pivot.

5. Method **`quickSort_1014()`** merupakan implementasi utama algoritma Quick Sort. Method ini bekerja secara rekursif dengan memanggil method `partition_1014()` untuk membagi array menjadi beberapa bagian yang lebih kecil, kemudian mengurutkan masing-masing bagian hingga seluruh data tersusun dengan benar berdasarkan durasi lagu secara ascending.

6. Method **`tampilData_1014()`** digunakan untuk menampilkan seluruh data lagu yang tersimpan pada array. Informasi yang ditampilkan meliputi judul lagu, penyanyi, dan durasi lagu sehingga pengguna dapat melihat kondisi data sebelum maupun sesudah proses sorting.

7. Pada method **`main()`**, program diawali dengan membuat objek `Sorting_2511531014`, kemudian pengguna diminta memasukkan data lagu. Setelah itu program menampilkan data sebelum sorting, menjalankan algoritma Quick Sort, dan menampilkan kembali data yang telah terurut berdasarkan durasi lagu dari yang terkecil ke terbesar.

8. Secara keseluruhan, program ini digunakan untuk memahami cara kerja algoritma **Quick Sort** dalam mengurutkan data objek pada array. Melalui program ini dapat dipahami proses partition, pertukaran data, rekursi, serta penerapan algoritma sorting dalam pengelolaan playlist lagu secara terstruktur dan efisien.

OUTPUT:


```
Console X
<terminated> Sorting_2511531014 [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (Jun 5, 2026, 8:34:59 PM - 8:36:52 PM elapsed: 0:01:53.108)
=== Sorting Playlist NIM: 2511531014 ===
Jumlah lagu (minimal 7): 7

Data Lagu ke-1
Judul : Jatuh suka
Penyanyi : Tulus
Durasi (detik) : 340

Data Lagu ke-2
Judul : Sepatu
Penyanyi : Tulus
Durasi (detik) : 230

Data Lagu ke-3
Judul : Tanggung Jawab
Penyanyi : Juicy Luicy
Durasi (detik) : 439

Data Lagu ke-4
Judul : Die With A Smile
Penyanyi : Lady Gaga & Bruno Mars
Durasi (detik) : 256

Data Lagu ke-5
Judul : Komang
Penyanyi : Raim Laode
Durasi (detik) : 224

Data Lagu ke-6
Judul : Monokrom
Penyanyi : Tulus
Durasi (detik) : 389

Data Lagu ke-7
Judul : Evaluasi
Penyanyi : Hindia
Durasi (detik) : 305
|
=== Data Sebelum Sorting ===
1. Jatuh suka - Tulus - 340 detik
2. Sepatu - Tulus - 230 detik
3. Tanggung Jawab - Juicy Luicy - 439 detik
4. Die With A Smile - Lady Gaga & Bruno Mars - 256 detik
5. Komang - Raim Laode - 224 detik
6. Monokrom - Tulus - 389 detik
7. Evaluasi - Hindia - 305 detik

=== Data Setelah Quick Sort (Durasi Asc) ===
1. Komang - Raim Laode - 224 detik
2. Sepatu - Tulus - 230 detik
3. Die With A Smile - Lady Gaga & Bruno Mars - 256 detik
4. Evaluasi - Hindia - 305 detik
5. Jatuh suka - Tulus - 340 detik
6. Monokrom - Tulus - 389 detik
7. Tanggung Jawab - Juicy Luicy - 439 detik
```